Introduction to Containers

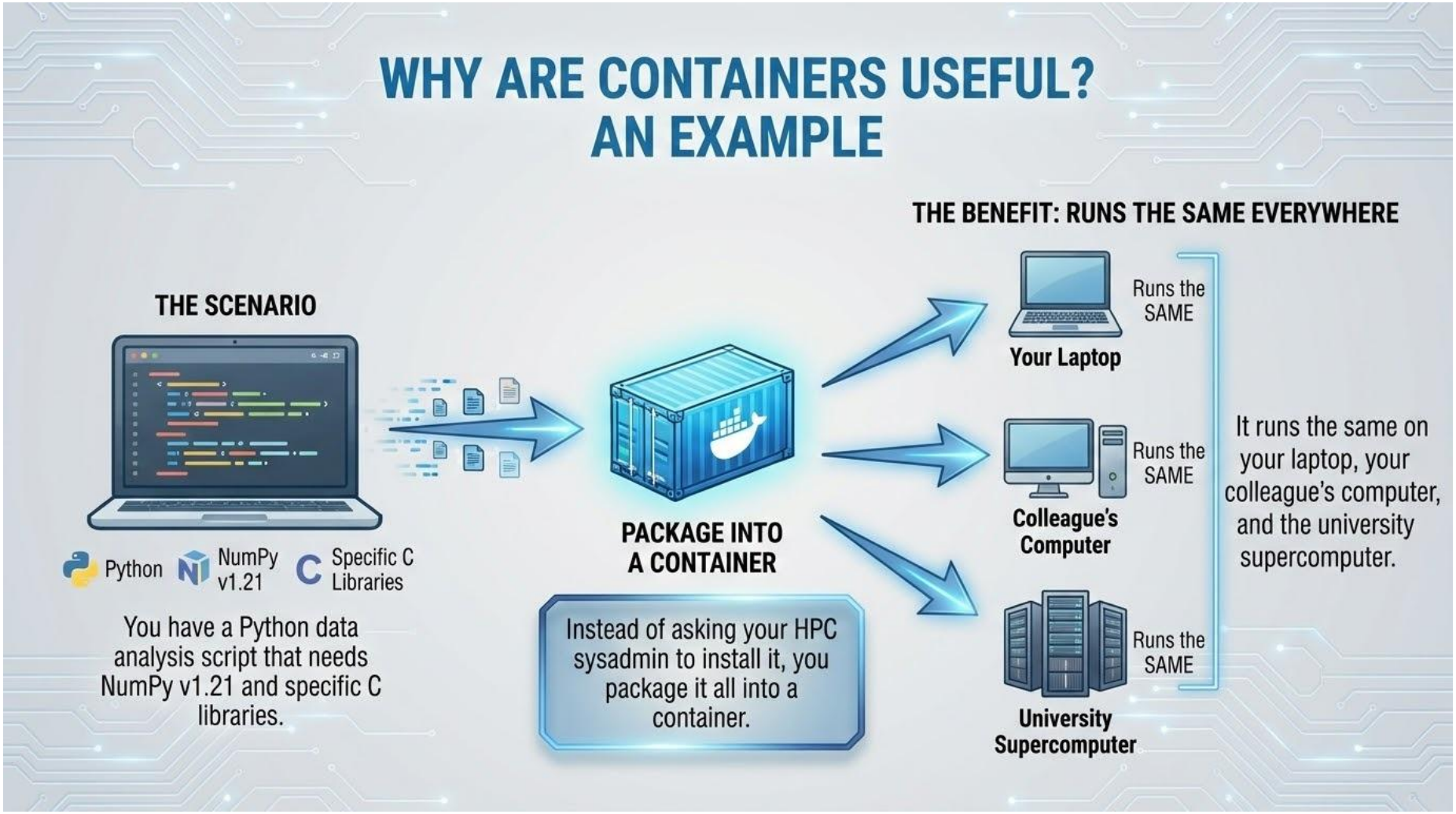
Singularity & Docker

11:30 – 13:00

What is a Container?

Lightweight package with application + dependencies runs consistently across laptops, servers, and clusters. Shares the host OS kernel (unlike virtual machines). Very fast to start and resource-efficient

What is a Container?



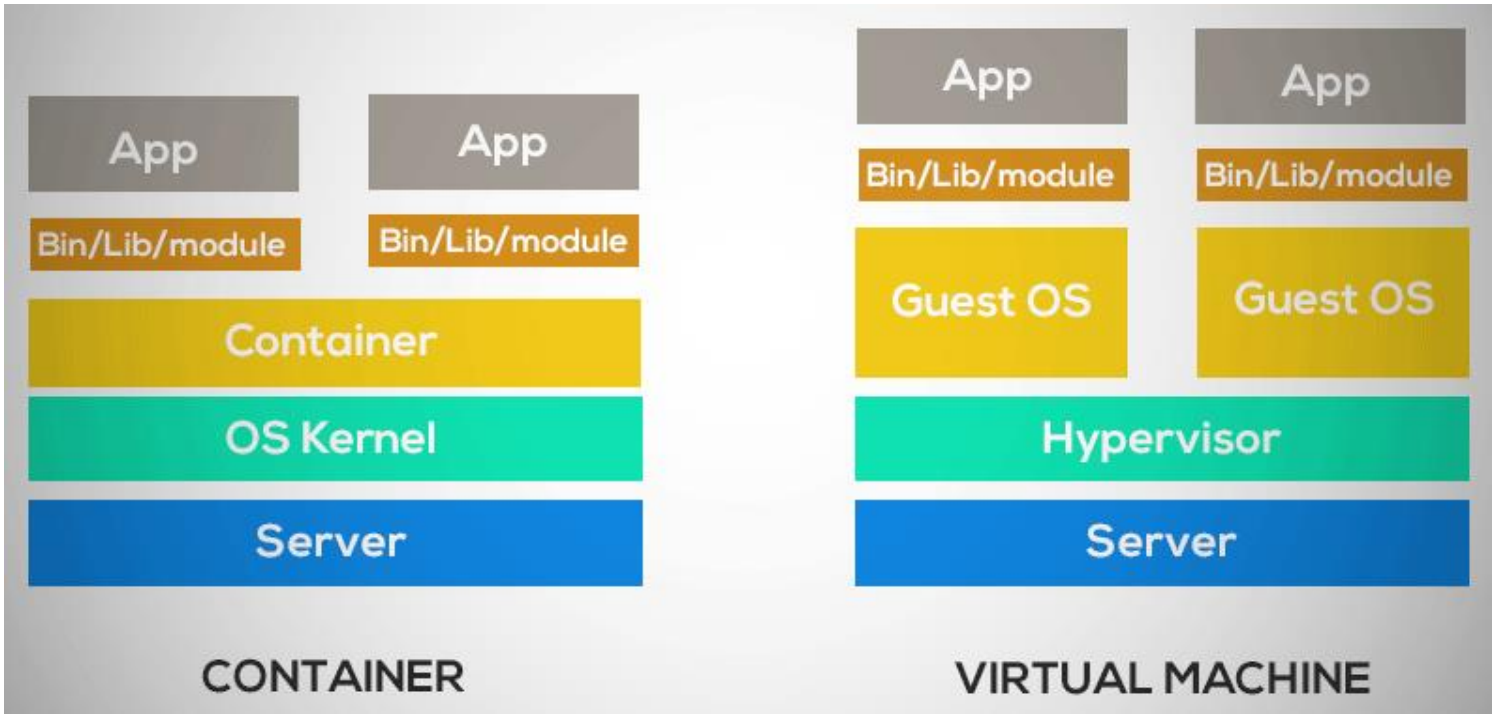
IMAGES VS RUNNING CONTAINERS

Container Image
An immutable, read-only file that contains the source code, libraries, dependencies, tools, and other files needed for an application to run. They may sometimes be referred to as a *disk image* or *container image* and they may be stored in different ways, perhaps as a single file, or as a group of files

Running Container
A live, running instance of an image. It adds a writable layer on top of the read-only image, allowing the application to execute and modify state temporarily. Isolated process with its own namespace. Ephemeral (changes lost unless mounted)

Containers vs Virtual Machines

Virtual Machines include a full operating system. Containers only include what the application needs and is built for a target OS/architecture. So, if you build an image that run on Windows, the container might not run on Mac OS or Linux without a VM. Compared to VM containers are smaller, faster and easier to scale



Docker vs Singularity

The industry standard

Target environment:

Cloud, Microservices, Web Servers, CI/CD pipelines.
Optimized for single-tenant or privileged environments.

Security model

Root Daemon (usually). Processes often run as root.
Requires privileged access to install/run.

Workflow and image

Layered OCI Images. Ephemeral containers.

```
docker build -t app .
docker run -v /data:/data app
```

HPC scientific computing

Target environment:

High Performance Computing (HPC), Shared Clusters
Optimized for multi-user systems.

Security model

User Space. Runs as the invoking user (no root daemon).
Same permissions as your user account.

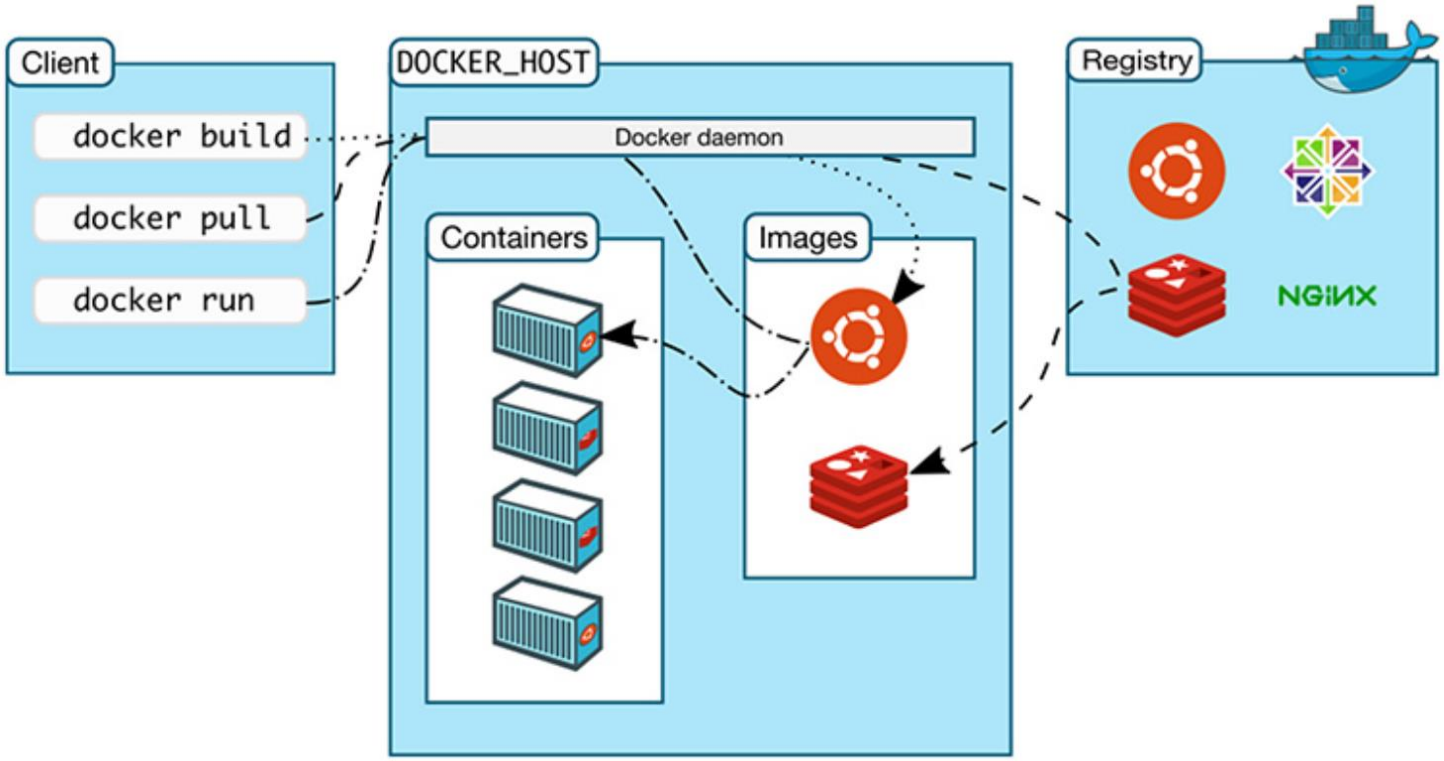
Workflow and image

Single File (SIF). Interoperable with Docker.

```
singularity pull docker://alpine
singularity exec image.sif app
```

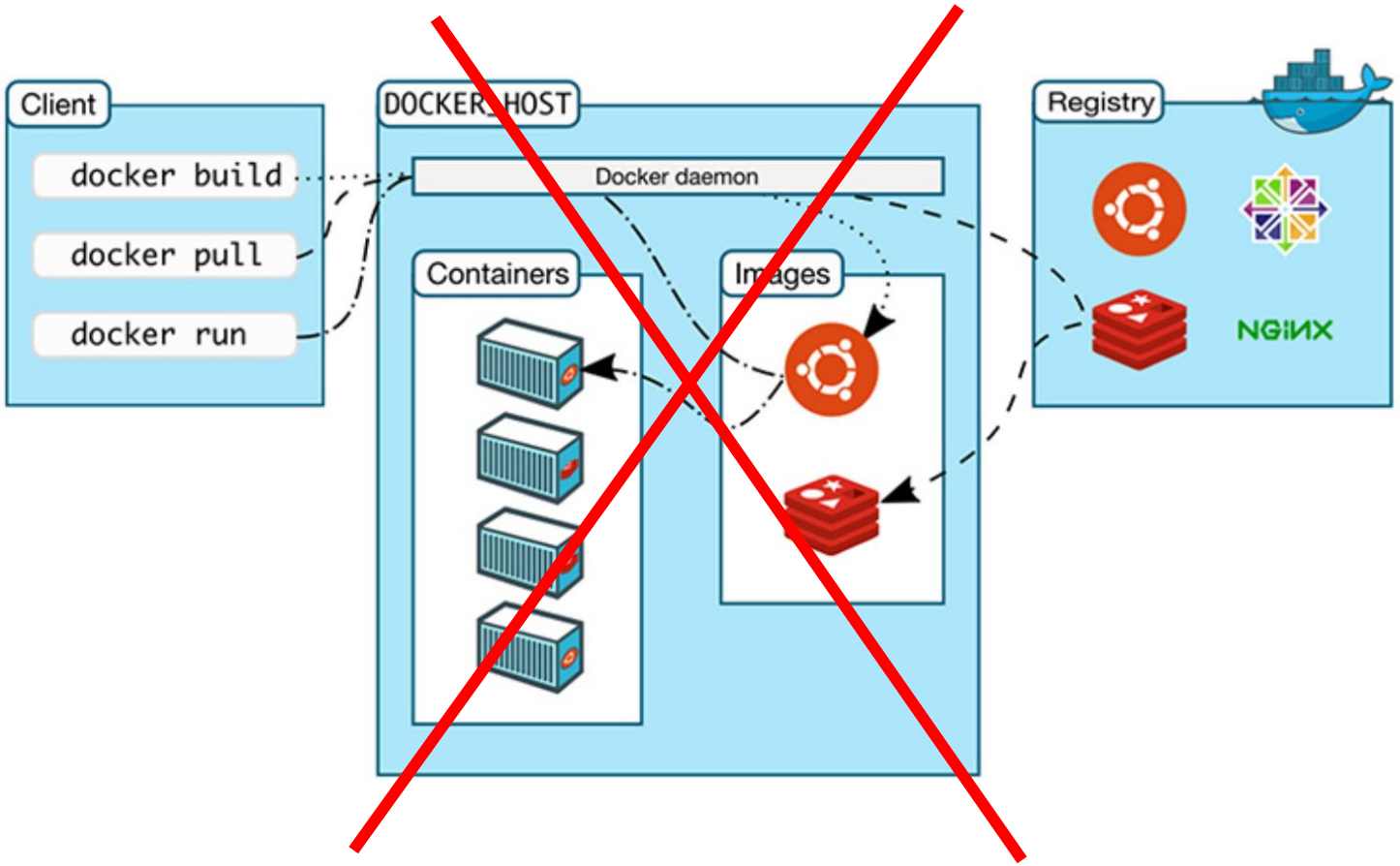
How usually a Docker container system works?

A service called Docker daemon orchestrate all the operations between the Docker host, the Client and the Registry.

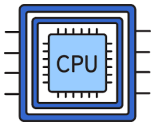


What about singularity?

In singularity we don’t have a daemon service that control that. We only have a binary and command line that the user interact with



Main concept: Isolation



Process Isolation

Docker: Containers use PID namespaces. It cannot see or signal processes on the host or in other containers.
Singularity: By default, it share the same PID space unless specified (--pid)

```
[~]bruno.ariano@hnode02$ singularity exec --pid docker://alpine ps -ef
INFO:    Using cached SIF image
PID   USER     TIME  COMMAND
    1 bruno.ar  0:00 sinit
   13 bruno.ar  0:00 /bin/ps -ef
```

Filesystem Isolation

Docker/Singularity: The container sees a unique root filesystem (from the image). Host files are invisible unless explicitly mounted (bind mounts). However, Singularity by default also mount the home and the current directory

Network Isolation

Docker: Own IP, bridge network by default.
Singularity: Shares host network by default (for HPC speed) but can be isolated with flags (only fake- root access).

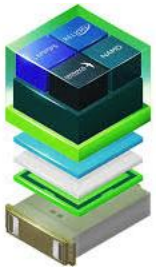
Singularity Isolation Demo

```
# Run with stricter isolation flags
$ singularity exec --contain --net docker://alpine ip addr

# Default behavior (shares host network)
$ singularity exec docker://alpine ip addr
(Shows same interfaces as the host machine)
```

Where to Get Containers

Docker Hub
The default & largest library. Most common source.

Quay.io
Red Hat supported. Home to Biocontainers.



GHCR & NVIDIA NGC
GitHub Registry & GPU-optimized catalogs.

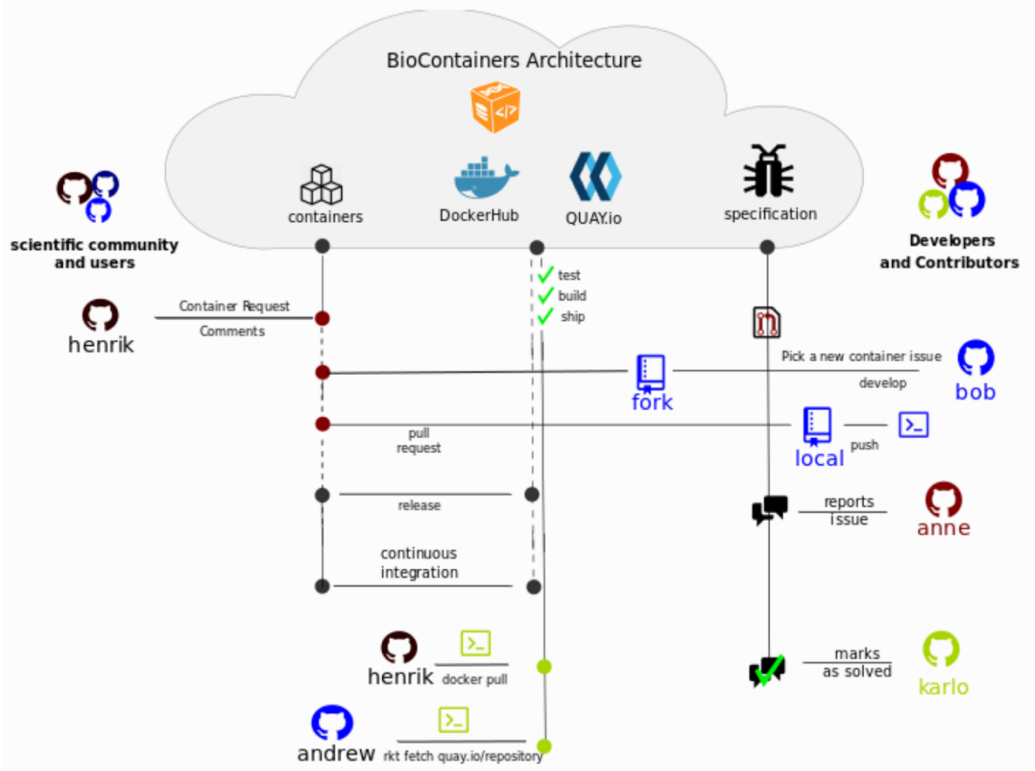
Sylabs Cloud
Native Singularity Image Format (SIF) library.

Terminal - Pull Commands

```
# 1. From Docker Hub (Standard OCI images)
$ singularity pull ubuntu_22.sif docker://ubuntu:22.04
  INFO: Converting OCI blobs to SIF format...

# 2. From Quay.io (Bioinformatics example)
$ singularity pull fastqc.sif
  docker://quay.io/biocontainers/fastqc:0.12.1--0

# 3. From GitHub Container Registry
$ singularity pull mytool.sif docker://ghcr.io/username/repo:tag

# 4. From Sylabs Cloud (Native SIF)
$ singularity pull lolcow.sif
  library://sylabsed/examples/lolcow:latest
```

Where to Get Bioinformatics Containers

BioContainers Project
A community-driven project that provides standardized containers for bioinformatics software (https://biocontainers.pro).



Best Practice: Pin Versions
Avoid using latest. Always specify the full version tag to ensure your analysis is reproducible years from now

Singularity CLI Basics

pull

Download Images

Fetches images from registries (Docker Hub, Quay, etc...) and converts them to SIF if necessary.

build

Create Images

Constructs a new container from a definition file. Usually this require root privileges

run/exec

Execute Commands

run: Launches a default command set at the beginning.
exec: Runs a specific custom command.

inspect

View Metadata

Shows labels, environment variables, and runscript details stored in the image.

shell

Open Shell

starts an interactive shell session inside the container

Docker Example

```
# 1. Get Image
$ docker pull python:3.11-slim
# 2. Run Container
$ docker run --rm python:3.11-slim python -V
```

Singularity Example

```
# 1. Pull an image from Docker Hub (creates alpine_3.20.sif)
$ singularity pull alpine_3.20.sif docker://alpine:3.20
INFO: Converting OCI blobs to SIF format...

# 2. Execute a specific command (ad-hoc)
$ singularity exec alpine_3.20.sif cat /etc/os-release

# 3. Enter interactive shell mode
$ singularity shell alpine_3.20.sif
Singularity> whoami
bruno

# 4. Inspect container labels
$ singularity inspect --labels alpine_3.20.sif
org.label-schema.name: Alpine Linux
org.label-schema.version: 3.20.0
```

Singularity: Mounting Data

The Bind Flag (-B / --bind)
Maps directories from host to container.
Syntax: src:dest[:opts]
Options: rw (default, read-write), ro (read-only).

Control Your Context
--pwd /path: Sets the initial working directory inside the container.
--home /path: Overrides the home directory mount (Singularity mounts $HOME by default).

Best Practices
Always use **absolute paths** to avoid confusion.
Use **read-only (ro)** mounts for raw input data to ensure reproducibility and prevent accidental deletion.

Complex Workflow Example

```
# Mount Project (RW), Data (Read-Only), and set WorkDir
$ singularity exec \
  > -B $PWD/project:/proj:rw \
  > -B /data/shared:/data:ro \
  > --pwd /proj \
  > analysis.sif python train.py --data /data

| Process runs in /proj. Can write to /proj. Can only read /data.
```

Comma-Separated Syntax

```
# You can list multiple binds separated by commas
$ singularity shell \
  > -B /ref:/ref:ro,/scratch:/scratch \
  > biotools.sif
```

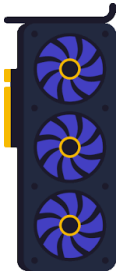
Singularity: GPU Access

How it Works

Unlike VMs, containers share the host kernel. To use GPUs, Singularity binds the host's driver libraries and device files (/dev/nvidia*) into the container at runtime.

NVIDIA GPU Workflow

```
# Check if GPU is visible inside container
$ singularity exec --nv docker://nvidia/cuda:11.0-base nvidia-smi

# Run PyTorch with GPU support
$ singularity exec --nv torch_latest.sif python training.py

# Limiting visible devices (Environment Var)
```



NVIDIA GPUs (--nv)

Use the --nv flag. This is the standard for most AI/HPC clusters. Requires NVIDIA drivers on host and CUDA toolkit in container (or compatible libs).

AMD ROCm Workflow

```
# Run container with ROCm support
$ singularity exec --rocm docker://rocm/pytorch:latest \
  python -c "import torch; print(torch.cuda.is_available())"
  True
```

AMD GPUs (--rocm)

Use the --rocm flag for AMD Radeon Instinct cards. Similar mechanism: binds ROCm libraries and devices into the container.

Summary & Key Takeaways

Core Concepts

- **Containers** package apps & dependencies for reproducibility and portability across systems.
- **Platform Choice:** Use Docker for cloud/services; use Singularity/Apptainer for HPC/shared clusters.
- **Isolation:** Containers isolate processes and filesystems but can access host data via bind mounts.
- **Registries:** Pull images from Docker Hub, Quay.io, GHCR, or Sylabs Cloud.

Singularity Essentials

- **Lifecycle:** pull images, then run / exec / shell them.
- **Data:** Use -B /path:/mnt to bind mount host directories.
- **Hardware:** Use --nv for NVIDIA GPU access or --rocm for AMD.
- **Config:** Use SINGULARITYENV_VAR=val to pass environment variables.

Next Steps

Practice: Try containerizing one of your own analysis scripts or workflows today.

Pin Versions: Always use specific tags (e.g., `python:3.9`) instead of `latest` for reproducibility.

Explore: Look into building your own images with Definition Files (Singularity) or Dockerfiles.